

Ouvidoria Inteligente — Triagem Semântica de Manifestações

Autor: PEDRO HENRIQUE BENICIO DE OLIVEIRA — RGM: 33602697

Relatório de decisões, dificuldades e aprendizados • Desafio Prático de NLP Aplicado

1. Visão geral

O protótipo substitui a busca por palavras-chave por representações vetoriais para (i) medir similaridade entre manifestações, (ii) detectar duplicatas, (iii) tratar textos longos com chunking e (iv) oferecer busca e visualização em um app Streamlit. O código compartilhado fica em **ouvidoria_utils.py** (carga de dados, embeddings, *detectar_duplicatas*, chunking, redução 2D) e é reutilizado pelos três notebooks e pelo app. Como o arquivo *manifestacoes.json* não acompanhava o enunciado, foi gerado um conjunto sintético de 40 manifestações (*gerar_dados.py*) seguindo a especificação: 5 categorias, 3 pares de duplicatas semânticas (6 manifestações = 15%) registrados em *duplicatas_reais.json* como gabarito, e 5 textos com 620–682 caracteres e vários assuntos cada.

2. Entrega 1 — Representações esparsas × densas

Comparamos BoW e TF-IDF (scikit-learn, 542 termos, 95% de esparsidade) com três modelos do sentence-transformers: paraphrase-multilingual-MiniLM-L12-v2, distiluse-base-multilingual-cased-v2 e multilingual-e5-small. O modelo *BAAI/bge-small-pt-v1.5* sugerido no enunciado não estava acessível no Hugging Face (erro 401) e foi substituído pelo distiluse.

Par	BoW	TF-IDF	MiniLM	distiluse	e5-small
M003 × M017 (duplicata)	0,00	0,00	0,48	0,25	0,87
M008 × M022 (duplicata)	0,16	0,07	0,80	0,56	0,90
M008 × M031 (sem relação)	0,00	0,00	0,10	0,14	0,89

Os modelos esparsos não distinguem uma duplicata sem palavras em comum de um par sem relação (ambos 0), pois "buraco" e "esburacado" são tokens distintos. Os embeddings densos separam bem o par de saúde (0,80 × 0,10 no MiniLM), mas o par do buraco ficou em 0,48, porque M003 tem detalhes que M017 não tem. O e5-small comprime todos os scores em 0,81–0,93 (média 0,87), tornando o cosseno pouco discriminativo sem recalibração: **o limiar não é transferível entre modelos**.

3. Entrega 2 — Duplicatas

A função *detectar_duplicatas(textos, limiar=0.85)* calcula a matriz de cosseno e retorna os pares acima do limiar. Com 0,85 e o MiniLM, apenas M012×M035 (0,89) é encontrado: precisão 100%, revocação 33% (2 falsos negativos: M008×M022 com 0,80 e M003×M017 com 0,48). O melhor F1 (0,67) ocorre perto de 0,775, com 1 falso positivo (M017×M020, mesmo tema de asfalto, mas manifestações diferentes). Nenhum limiar separa perfeitamente as classes: o maior cosseno entre não duplicatas (0,80) é maior que o menor entre duplicatas (0,48). Decisão: **0,85 para fusão automática** (evita descartar manifestações legítimas) e **0,75–0,78 como fila de revisão humana**. O percentil 99 da distribuição (0,67) serve como limiar dinâmico para ordenar candidatos. O gabarito tem só 3 pares, então as métricas são instáveis.

4. Entrega 3 — Chunking

Usamos *RecursiveCharacterTextSplitter* com a hierarquia de separadores parágrafo, linha, frase, ponto e vírgula, vírgula e espaço, mantendo o separador ao fim do trecho. Testamos quatro configurações nas 5 manifestações mais longas:

chunk_size / overlap	nº chunks	sim. consecutivos	sim. não vizinhos	vizinho na mesma manifestação
200 / 0	25	0,355	0,326	52%

chunk_size / overlap	nº chunks	sim. consecutivos	sim. não vizinhos	vizinho na mesma manifestação
200 / 50	25	0,355	0,326	52%
400 / 0	11	0,514	0,382	82%
400 / 150	14	0,655	0,500	93%

Achado importante: o overlap só tem efeito quando cabe pelo menos uma unidade de corte (uma frase de ~125 caracteres) dentro dele e do chunk; por isso 200/50 é idêntico a 200/0. A configuração **400/150** preservou melhor o sentido: a frase de fronteira aparece nos dois chunks, mantendo ligados problemas encadeados (bueiros entupidos → alagamento) e elevando a coesão entre vizinhos (0,51 → 0,65). O custo é cerca de 27% mais chunks. No espaço 2D (PCA e t-SNE), chunks da mesma manifestação tendem a ficar próximos, mas não em clusters compactos, pois os textos são multi-assunto.

5. Entrega 4 — App Streamlit

app_ouvidoria.py tem sidebar com seleção de modelo e top-k, e quatro abas: **Busca Semântica** (top-k com cores ■ > 0,7, ■ > 0,5, ■ demais); **Base Completa** (tabela filtrável e botão que gera o heatmap e os pares mais similares); **Espaço Vetorial** (PCA/t-SNE por categoria, com a discussão dos clusters); **Chunking** (texto livre ou exemplo, estratégia Recursive/Sentenças/Caracteres, parâmetros, chunks, embeddings, matriz e projeção). O modelo usa *st.cache_resource* e os embeddings, *st.cache_data*. Medimos que, com o MiniLM, o vizinho mais próximo tem a mesma categoria oficial em 80% dos casos (saúde 100%, segurança 57%): os clusters coincidem apenas em parte, pois os embeddings agrupam por tema, e a categoria oficial reflete a secretaria responsável.

6. Dificuldades

- Dados ausentes: o JSON foi recriado de forma sintética, então os resultados numéricos valem para este conjunto.
- Modelo bge-small-pt inacessível: substituído.
- Scores do e5 comprimidos: exigem prefixo *query*: e calibração.
- O overlap do splitter recursivo não age em qualquer combinação de parâmetros.
- Gabarito pequeno (3 pares): precisão e revocação variam muito com um único par.

7. Aprendizados

1) Embeddings resolvem o problema de vocabulário, mas exigem calibração de limiar por modelo. 2) Mesmo tema não é o mesmo problema: duplicatas devem ser confirmadas por humanos ou por regras adicionais (local, data). 3) A escolha de chunking deve seguir a estrutura do texto (frases) e não apenas números arbitrários. 4) Métricas quantitativas (vizinho mais próximo, silhouette, F1) são mais confiáveis do que a inspeção visual de projeções 2D. 5) Próximos passos: avaliar modelos maiores (bge-m3), busca híbrida BM25 + vetorial e um gabarito maior.

8. Como executar

`pip install -r requirements.txt → streamlit run app_ouvidoria.py`. Notebooks: *analise_comparativa*, *deteccao_duplicatas* e *chunking_manifestacoes*. `python gerar_notebooks.py`
 --executar os regenera.